

Learning STARK

Safe Systems Programming for Typed ML Deployment

CHAPTER 02

A Guided Tour of a STARK Program

Learning STARK

Chapter 2 preview: *A Guided Tour of a STARK Program*

Early Access manuscript - July 2026

This chapter documents a language specification under active implementation. Core v1 is a normative draft and has not yet been validated by a conforming compiler. Illustrative diagnostics are identified in the text.

The design and text are original to the STARK Language Project. Product and project names mentioned for interoperability remain the property of their respective owners.

Source manuscript: `book/manuscript/ch02-guided-tour.md`

A Guided Tour of a STARK Program

A language begins to feel real when its rules stop being a list and start working together on one page.

Chapter 1 described STARK's argument: important deployment constraints should be visible to the compiler, dynamic evidence should be validated at explicit boundaries, and the language should build on a small memory-safe core. This chapter turns from the argument to the language itself.

We will read a complete Core v1 program that validates an image size before later processing begins. The example contains no tensor-extension syntax. It uses ordinary language features that every future STARK deployment program will depend on: structs, enums, functions, methods, local inference, explicit signatures, borrowing, `Result`, `match`, and expression-valued blocks.

The goal is not to memorize every token. It is to develop a reliable way to read STARK source. By the end of the chapter, you should be able to identify the top-level items in a file, follow ownership across a function call, distinguish a statement from a value-producing expression, and interpret the shape of a compiler diagnostic.

Reading code in a specification-stage project

The examples in this chapter follow the normative Core v1 draft. They cannot yet be executed by a conforming STARK compiler because that compiler does not exist. The examples are language-definition material, not evidence of an implemented toolchain.

2.1 The program we will explore

Suppose an image service must reject empty images and inputs whose pixel count exceeds a configured limit. This is intentionally simpler than model inference. It lets us see the language's general-purpose foundation without mixing in symbolic tensor dimensions or artifact loading.

STARK

```
// Core v1
struct ImageSize {
    width: UInt32,
    height: UInt32
}

enum SizeError {
    ZeroWidth,
    ZeroHeight,
    TooManyPixels
}

impl ImageSize {
    fn new(width: UInt32, height: UInt32) -> ImageSize {
        ImageSize { width: width, height: height }
    }

    fn area(&self) -> UInt64 {
        let width = self.width as UInt64;
        let height = self.height as UInt64;
        width * height
    }
}
```

STARK

```
// Core v1 - program continued
fn validate_size(
    size: &ImageSize,
    max_pixels: UInt64
) -> Result<UInt64, SizeError> {
    if size.width == 0u32 {
        return Result::Err(SizeError::ZeroWidth);
    }

    if size.height == 0u32 {
        return Result::Err(SizeError::ZeroHeight);
    }

    let pixels = size.area();

    if pixels > max_pixels {
        Result::Err(SizeError::TooManyPixels)
    } else {
        Result::Ok(pixels)
    }
}

fn main() {
    let size = ImageSize::new(1920u32, 1080u32);
    let max_pixels: UInt64 = 4_000_000u64;
    let outcome = validate_size(&size, max_pixels);

    match outcome {
        Result::Ok(_) => println("image accepted"),
        Result::Err(SizeError::ZeroWidth) => println("width must be positive"),
        Result::Err(SizeError::ZeroHeight) => println("height must be positive"),
        Result::Err(SizeError::TooManyPixels) => println("image is too large")
    }
}
```

Even before studying the details, you can read the program's outline. `ImageSize` groups a width and height. `SizeError` lists three reasons validation may fail. An `impl` block attaches construction and area calculation to `ImageSize`. `validate_size` borrows an image size, compares it with a limit, and returns either a pixel count or a typed error. `main` creates values, calls validation, and handles every possible result.

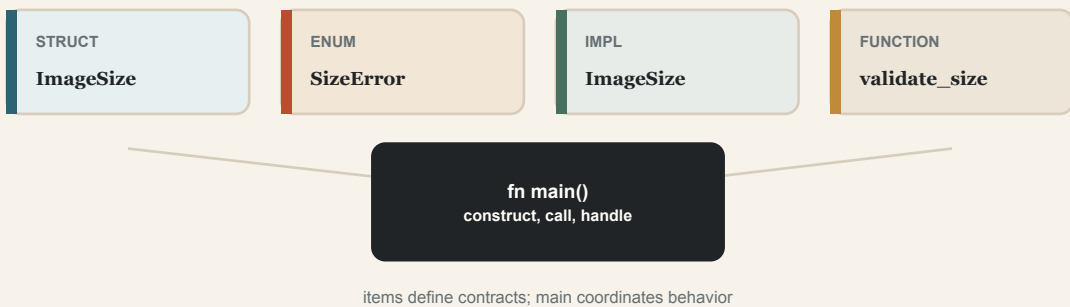


Figure 2-1. Top-level items establish the types and functions coordinated by main.

That outline is the first reading skill to cultivate: identify declarations before tracing expressions. A STARK source file contains top-level **items**. Items establish the names and contracts used inside function bodies. Once those contracts are visible, the body of `main` becomes much easier to follow.

2.2 Source files are sequences of items

The Core v1 grammar defines a program as zero or more items. A source file does not permit arbitrary executable statements at module scope. Computation belongs inside functions, method bodies, constant expressions, and other places admitted by the grammar.

Core v1 items include:

- functions declared with `fn`;
- structs declared with `struct`;
- enums declared with `enum`;
- traits and implementations declared with `trait` and `impl`;
- compile-time constants declared with `const`;
- type aliases declared with `type`;
- imports declared with `use`; and
- modules declared with `mod`.

An item may be public or private. The default visibility rules and module boundaries are covered later in the book; for now, notice that the fields in `ImageSize` are private because they carry no `pub` modifier.

Order is primarily for the reader. Top-level symbol construction gives functions access to items in their module even when a declaration appears later in the file. A well-organized file still tends

to place data types before their implementations and entry points after the helpers they coordinate.

Names and case

Identifiers begin with a letter or underscore and may contain letters, digits, and underscores. They are case-sensitive. The language specification does not turn naming style into grammar, but the examples follow familiar conventions:

- `ImageSize` and `SizeError` use PascalCase for types.
- `validate_size` and `max_pixels` use snake_case for functions and values.
- `TooManyPixels` uses PascalCase for an enum variant.
- Type names such as `UInt32` are keywords, not ordinary identifiers.

`size`, `Size`, and `SIZE` would be three different names. Keywords such as `fn`, `match`, and `return` cannot be reused as identifiers. Several additional words, including `async`, `await`, `unsafe`, and `import`, are reserved for future language work and are rejected even though Core v1 does not yet assign them grammar productions.

Comments and whitespace

`//` begins a line comment. `/* ... */` creates a block comment, and block comments may nest. Documentation comments begin with `/**` and attach to the following declaration.

Whitespace separates tokens but does not terminate statements. Newlines make source readable; semicolons carry the syntactic meaning. The following two bindings are equivalent to the parser:

STARK

```
// Core v1
let width = 1920u32;
let height = 1080u32;
```

STARK

```
// Core v1 - legal, but hostile to readers
let width = 1920u32; let height = 1080u32;
```

Production style should use formatting to expose structure even when the grammar does not require it.

2.3 `main` is an ordinary function with a special role

Execution begins at `main`, but `main` uses the ordinary function syntax:

STARK

```
// Core v1
fn main() {
    println("hello from STARK");
}
```

The keyword `fn` introduces a function. `main` is its name. Empty parentheses mean it takes no parameters. Because there is no `->` annotation, the function returns `Unit`, the type used when a computation has no meaningful value to return.

Core v1 function return types are never inferred. A function that returns a value must say so:

STARK

```
// Core v1
fn default_limit() -> UInt64 {
    4_000_000u64
}
```

The body is a block. The final expression has no semicolon, so its value becomes the value of the block and therefore the function result. There is no explicit `return` in this small function.

An explicit return is also legal:

STARK

```
// Core v1
fn default_limit() -> UInt64 {
    return 4_000_000u64;
}
```

The two functions have the same result. Idiomatic expression-oriented code usually reserves `return` for early exits and lets the final expression produce the normal result. Our `validate_size` function follows that pattern: error cases return early, while the final `if` expression produces either `Ok` or `Err`.

Entrypoint details are intentionally narrow here

Core v1 defines ordinary function and return semantics. Toolchain-level questions such as command-line arguments, process environment, executable packaging, and alternative entrypoints belong to implementation and platform documentation that has not yet been finalized.

2.4 Bindings are immutable unless marked otherwise

The first three lines of `main` introduce local bindings:

STARK

```
// Core v1
let size = ImageSize::new(1920u32, 1080u32);
let max_pixels: UInt64 = 4_000_000u64;
let outcome = validate_size(&size, max_pixels);
```

`let` creates a binding in the current lexical scope. A binding is immutable by default. After `size` has been initialized, the program cannot assign a different `ImageSize` to it or change one of its fields through that binding.

Mutability must appear in the declaration:

STARK

```
// Core v1
let mut attempts: UInt32 = 1u32;
attempts += 1u32;
```

`mut` does not mean that every value reachable from `attempts` is globally mutable. It grants mutable access through this binding, subject to ownership and borrowing rules. Those rules become more important for compound values and references.

Annotation and inference

The three bindings show two forms:

STARK

```
let name = expression;
let name: Type = expression;
```

The compiler infers the first form from the initializer. The second form states the required type and checks that the initializer is compatible.

In the example, the return type of `ImageSize::new` determines the type of `size`. The explicit `UInt64` annotation on `max_pixels` makes the intended numeric width visible to readers, though the `u64` suffix already fixes the literal's type. The return signature of `validate_size` determines that `outcome` is `Result<UInt64, SizeError>`.

Inference is local. It removes repetition; it does not make public function contracts implicit. Parameters and non-`Unit` return values remain explicitly typed.

Literal types

The suffixes `u32` and `u64` fix integer literal types:

STARK

```
1920u32      // UInt32
4_000_000u64 // UInt64
```

Underscores improve readability and do not change the value. They may occur only between digits. `4__000`, `_4000`, and `4000_` are invalid numeric literals.

An unsuffixed integer literal defaults to `Int32` when the value fits and to `Int64` otherwise. An unsuffixed floating-point literal defaults to `Float64`. STARK does not silently convert one numeric type into another merely because a value happens to fit.

Inference removes noise, not contracts

Infer a local value when its type is evident from nearby code. State a type when it defines an interface, controls representation, or prevents a reader from having to reconstruct intent.

2.5 Statements perform work; expressions produce values

The difference between a statement and an expression is central to reading STARK.

A statement occupies a position in a block and does not contribute a value to a surrounding expression. Bindings, explicit returns, breaks, continues, and expressions followed by semicolons are statements.

An expression produces a value. Literals, function calls, arithmetic, struct construction, `if`, `match`, loops, and blocks are expressions. An expression can be turned into a statement by adding a semicolon and discarding its value.

Compare these blocks:

STARK

```
// Core v1
fn area_a(width: UInt64, height: UInt64) -> UInt64 {
    width * height
}

fn area_b(width: UInt64, height: UInt64) {
    width * height;
}
```

`area_a` returns `UInt64`. Its final multiplication has no semicolon and is the block value. `area_b` returns `Unit`. The semicolon turns the multiplication into an expression statement whose result is discarded.

VALUE

```
width * height
```

block type: `UInt64`

STATEMENT

```
width * height;
```

block type: `Unit`

Figure 2-2. The final semicolon determines whether a block returns a value or `Unit`.

This rule explains why semicolon mistakes can produce type errors that initially seem unrelated. If a function promises `UInt64` but its last expression is accidentally terminated, its body evaluates to `Unit`.

TEXT

```
error[E0001]: function body has incompatible type
  --> image_size.stark:3:5
  |
1 | fn area(width: UInt64, height: UInt64) -> UInt64 {
  |                                           ----- required UInt64
2 |     width * height;
  |     ^^^^^^^^^^^^^^^^^ this statement discards a UInt64 value
3 | }
  | ^ body evaluates to Unit
  |
help: remove the semicolon to return the multiplication result
```

This diagnostic is illustrative. The exact rendering belongs to the future compiler, but the Core semantic-analysis specification requires structured source locations, error categories, and actionable suggestions.

2.6 Structs give related values one type

`ImageSize` is a struct:

STARK

```
// Core v1
struct ImageSize {
    width: UInt32,
    height: UInt32
}
```

A struct defines a product type: every `ImageSize` contains both fields. Field declarations use `name: Type`. Commas separate fields. The trailing comma is optional, though many formatters prefer it for multi-line declarations because later additions produce smaller diffs.

A struct value is constructed with a path followed by field initializers:

STARK

```
// Core v1
let size = ImageSize {
    width: 1920u32,
    height: 1080u32
};
```

The fields can appear in either order, but each required field must be initialized exactly once and the value must match the declared type.

When a local binding has the same name as a field, shorthand removes repetition:

STARK

```
// Core v1
fn make_size(width: UInt32, height: UInt32) -> ImageSize {
    ImageSize { width, height }
}
```

`ImageSize { width, height }` means `ImageSize { width: width, height: height }`. The longer form used in the main example is more explicit for a first reading; both are normative Core syntax.

Field access uses a dot:

STARK

```
let width = size.width;
```

Whether access copies, moves, or borrows depends on the field type and context. `UInt32` implements `Copy`, so reading `size.width` copies the small scalar and leaves the struct usable. Reading a move-only field by value may partially move the struct, a topic covered with ownership in later chapters.

Core v1 does not permit a struct or enum declaration to write a direct reference field such as `field: &T`. The initial lifetime system is deliberately conservative. Generic types instantiated with references, such as `Option<&T>`, are allowed but treated as borrow-carrying values.

2.7 `impl` attaches behavior to a type

An implementation block groups functions associated with a type:

STARK

```
// Core v1
impl ImageSize {
    fn new(width: UInt32, height: UInt32) -> ImageSize {
        ImageSize { width, height }
    }

    fn area(&self) -> UInt64 {
        let width = self.width as UInt64;
        let height = self.height as UInt64;
        width * height
    }
}
```

`new` has no receiver parameter. It is called through the type path as `ImageSize::new(...)`. The name `new` has no magical allocation semantics; it is an ordinary associated function chosen by convention.

`area` has a receiver, `&self`. It is called through a value as `size.area()`. The receiver is an immutable borrow of the `ImageSize`, so calculating the area does not take ownership and cannot mutate the original value.

Core v1 permits three receiver forms inside traits and implementation blocks:

- `self` takes ownership of the receiver;
- `&self` borrows it immutably; and
- `&mut self` borrows it mutably.

The signature tells a caller about lifecycle as well as data. A consuming method may render a move-only value unavailable afterward. A shared-borrow method can be called while the owner remains in scope. A mutable-borrow method requires exclusive mutable access for the borrow's duration.

Explicit conversion

The `area` method converts each `UInt32` field to `UInt64`:

STARK

```
let width = self.width as UInt64;
let height = self.height as UInt64;
```

The `as` operator is visible because Core v1 defines no implicit numeric conversion. Multiplication then occurs between two `UInt64` values and returns `UInt64`.

Widening here also makes the area calculation less likely to overflow than multiplying in `UInt32`. It does not make overflow impossible. Core v1 integer overflow traps at runtime in every build profile. If dimensions could approach the limits of `UInt32`, the program would need a checked arithmetic policy rather than assuming that `UInt64` is always sufficient.

2.8 Borrowing preserves the caller's ownership

The validation function accepts `&ImageSize`:

STARK

```
// Core v1
fn validate_size(
    size: &ImageSize,
    max_pixels: UInt64
) -> Result<UInt64, SizeError> {
    // ...
}
```

At the call site, `&size` creates an immutable borrow:

STARK

```
let outcome = validate_size(&size, max_pixels);
```

The caller owns `size`. The function receives temporary permission to inspect it. When the call completes, the borrow ends and the caller still owns the original `ImageSize`.

`max_pixels` is passed by value. Because `UInt64` is a `Copy` type, passing it copies the scalar. The original binding remains usable. A move-only value such as `String`, `Vec<T>`, or a future `Tensor` would move when passed by value unless the parameter accepted a reference.

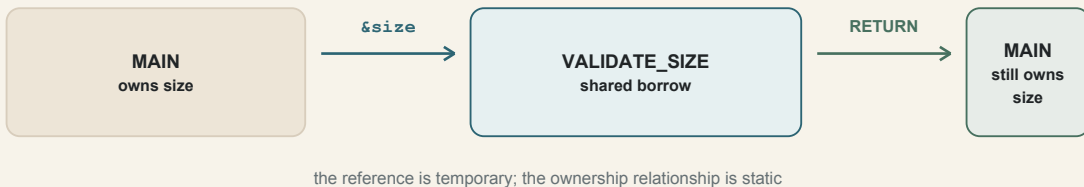


Figure 2-3. `&size` lends access for the call while ownership remains in main.

This is why signatures deserve careful reading. Compare:

STARK

```

fn inspect(size: &ImageSize)      // shared borrow
fn adjust(size: &mut ImageSize)   // exclusive mutable borrow
fn consume(size: ImageSize)       // ownership transfer
  
```

All three functions receive an image size, but they make different promises. The first cannot mutate or retain an invalid reference after the call. The second may mutate and requires exclusive access. The third becomes responsible for the value and its eventual cleanup.

Core v1 checks these relationships statically. Multiple immutable borrows may coexist. A mutable borrow cannot coexist with another active borrow of the same value. A reference cannot outlive the value it refers to. The current lifetime model is lexical and annotation-free, intentionally accepting a smaller set of patterns than Rust's full system.

2.9 Enums describe alternatives

`SizeError` is an enum with three unit variants:

STARK

```

// Core v1
enum SizeError {
    ZeroWidth,
    ZeroHeight,
    TooManyPixels
}
  
```


An enum value is exactly one of its variants. Unit variants carry no additional data. Tuple variants can carry unnamed fields, and struct variants can carry named fields:

STARK

```
// Core v1
enum ValidationIssue {
    Missing,
    OutsideRange(UInt64, UInt64),
    InvalidField { name: String }
}
```

Enums let the type system distinguish alternatives that might otherwise be represented by sentinel integers, null values, or loosely related strings.

`Result<T, E>` is itself a generic enum supplied by the standard library:

STARK

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}
```

Our validation function returns `Result<UInt64, SizeError>`. Success carries a `UInt64` pixel count. Failure carries one of the `SizeError` variants. A caller cannot receive a bare count and separately wonder whether an error flag was checked.

Early return and final expression

The first two checks use explicit early returns:

STARK

```
if size.width == 0u32 {
    return Result::Err(SizeError::ZeroWidth);
}
```

`return` exits the enclosing function. The final decision instead uses an `if` expression:

STARK

```
if pixels > max_pixels {
    Result::Err(SizeError::TooManyPixels)
} else {
    Result::Ok(pixels)
}
```

Both branches produce `Result<UInt64, SizeError>`, so the entire `if` expression has that type. Because it is the final expression in the function body, it becomes the function's result.

This expression orientation reduces temporary variables without hiding control flow. It also gives the type checker a precise rule: value-producing branches must agree on a type, allowing only defined coercions such as the never type `!` flowing into another expected type.

2.10 `match` makes handling explicit

`main` handles the validation result with `match`:

STARK

```
// Core v1
match outcome {
    Result::Ok(_) => println("image accepted"),
    Result::Err(SizeError::ZeroWidth) => println("width must be positive"),
    Result::Err(SizeError::ZeroHeight) => println("height must be positive"),
    Result::Err(SizeError::TooManyPixels) => println("image is too large")
}
```

The expression after `match` is the scrutinee. Each arm contains a pattern, `=>`, and an expression. Commas between arms are optional according to the grammar, but using them consistently improves readability.

Patterns can destructure nested values. `Result::Err(SizeError::ZeroWidth)` first selects the `Err` variant of `Result`, then matches the `ZeroWidth` variant stored inside it.

The underscore in `Result::Ok(_)` is a wildcard. It matches the successful pixel count without binding a name because this program only prints a fixed message. If the value were needed, the pattern could bind it:

STARK

```
Result::Ok(pixels) => use_pixel_count(pixels)
```

Core semantic analysis requires matches to be exhaustive. The four arms cover `Ok` and every possible `SizeError` inside `Err`. If a new error variant is added, the compiler should identify matches that no longer cover the enum.

TEXT

```
error[E0303]: non-exhaustive match
  --> main.stark:8:5
   |
  8 |     match outcome {
   |       ^^^^^^^^^^^ missing pattern:
   |                       Result::Err(SizeError::UnsupportedFormat)
   |
help: add an arm for the missing variant or an intentional wildcard
```

A wildcard can be useful when all remaining cases genuinely share behavior, but it also suppresses future exhaustiveness guidance. In boundary code, naming each failure often produces a more maintainable contract.

2.11 Paths tell you where a name comes from

STARK uses `::` to form paths:

STARK

```
ImageSize::new
SizeError::ZeroWidth
Result::Ok
```

The path on the left supplies context. `ImageSize::new` is an associated function. `SizeError::ZeroWidth` is an enum variant. `Result::Ok` is a generic enum constructor whose type arguments are inferred from context.

The dot operator has a different job:

STARK

```
size.width
size.area()
```

It accesses a field or performs method lookup on a value. When you read unfamiliar STARK code, `::` usually moves through modules, types, or associated items; `.` starts from a value.

Generic arguments in expression position use the explicit `::<...>` form when inference cannot determine them:

STARK

```
let bytes = size_of::<UInt64>();
```

This form avoids an ambiguity between generic brackets and the less-than operator. A bare `<` after an identifier in expression position is always relational syntax.

2.12 Reading a diagnostic as structured evidence

A compiler diagnostic should connect a violated rule to the source that produced it. The semantic-analysis specification defines a general shape:

TEXT

```
error[E0001]: type mismatch
--> src/main.stark:12:9
|
12 |     let max_pixels: UInt32 = 4_000_000u64;
|                               ^^^^^^^^^^^^^^^ expected UInt32, found UInt64
|                               |
|                               required by this annotation
|
help: use a UInt32 literal or change the binding type
```

error[E0001]: type mismatch

```
--> src/main.stark:12:9

12 | let max_pixels: UInt32 = 4_000_000u64;
    |           ^^^^^ ^^^^^^^^^^^^^^^
    |           |
    |           required here           found UInt64
```

help: align the annotation and literal type

Figure 2-4. A useful diagnostic connects the failed expression to the contract that required something else.

Read the message in layers:

1. **Category and code.** `error[E0001]` identifies a stable class that tests and documentation can reference.
2. **Summary.** "type mismatch" states the violated rule without guessing at intent.
3. **Primary location.** The arrow and caret identify the expression that failed.
4. **Related location.** The annotation explains where the expected type came from.
5. **Evidence.** The message states both types instead of saying only "invalid value."
6. **Repair.** The help text offers alternatives when the change is mechanical.

Not every error has one obvious fix. A borrow conflict may involve several uses. A missing match arm may require a product decision. A good diagnostic presents enough provenance for the programmer to choose rather than hiding uncertainty behind a confident but arbitrary rewrite.

The compiler implementation plan adopts source spans and structured diagnostics from its first gate. This is important because later analyses depend on the same foundation. A tensor-shape message can trace symbolic dimensions only if the compiler has preserved where declarations and operations originated.

2.13 The compiler's view of the program

Humans see declarations and intent. A compiler processes the file in stages.

First, the lexer turns UTF-8 source into tokens. It distinguishes keywords, identifiers, literals, punctuation, and operators. It applies maximal munch, so `>>=` becomes the longest valid operator rather than three unrelated tokens. Comments and insignificant whitespace do not enter the ordinary token stream.

Second, the parser checks the syntax grammar and builds an abstract syntax tree. It knows that `fn` begins a function, that `ImageSize { ... }` is a struct literal, and that multiplication binds more tightly than addition. The planned parser uses recursive descent for language structure and Pratt parsing for expression precedence.

Third, name resolution connects uses to declarations. It determines which `size`, `Result`, and `println` each path denotes. Lexical scopes are searched from the innermost block outward, followed by module items, imports, and prelude names.

Fourth, type checking determines expression types and validates calls, branches, patterns, conversions, and assignments. It infers local bindings from nearby constraints while enforcing explicit function signatures.

Fifth, ownership and borrow analysis tracks moves, shared borrows, mutable borrows, initialization, and use after move. Control-flow analysis verifies return paths and reachability. Pattern analysis checks exhaustiveness.

Only a program that passes these front-end stages is eligible for execution or lowering. The current roadmap implements them before tensor-specific semantics so that extension errors rest on a tested language foundation.

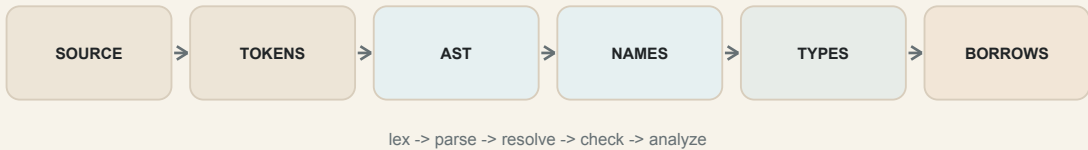


Figure 2-5. Each front-end stage adds meaning while preserving source provenance.

This pipeline also explains why one source mistake can produce several apparent symptoms. A missing delimiter may prevent the parser from seeing an entire item. An unresolved name may prevent type checking at its uses. Error recovery should continue far enough to report independent problems without flooding the programmer with consequences of the first defect.

2.14 The complete program, annotated

Now return to the whole program with line numbers. The numbers are editorial aids, not part of the source.

TEXT

```
01 | struct ImageSize {
02 |     width: UInt32,
03 |     height: UInt32
04 | }
05 |
06 | enum SizeError {
07 |     ZeroWidth,
08 |     ZeroHeight,
09 |     TooManyPixels
10 | }
11 |
12 | impl ImageSize {
13 |     fn new(width: UInt32, height: UInt32) -> ImageSize {
14 |         ImageSize { width: width, height: height }
15 |     }
16 |
17 |     fn area(&self) -> UInt64 {
18 |         let width = self.width as UInt64;
19 |         let height = self.height as UInt64;
20 |         width * height
21 |     }
22 | }
23 |
```

Lines 1-4 establish the valid representation of an image size. Width and height cannot accidentally exchange types because both are named fields, though a caller could still place the wrong numeric value in the wrong field. Types enforce structure, not intent that has no representation.

Lines 6-10 establish the complete error vocabulary for this validation policy. The enum is small enough that downstream handling can remain exhaustive.

Lines 12-22 attach two behaviors. `new` constructs an owned value. `area` borrows that value and returns a wider integer. The missing semicolon on line 20 is semantically significant.

TEXT

```

    | ... program continued ...
24 | fn validate_size(
25 |     size: &ImageSize,
26 |     max_pixels: UInt64
27 | ) -> Result<UInt64, SizeError> {
28 |     if size.width == 0u32 {
29 |         return Result::Err(SizeError::ZeroWidth);
30 |     }
31 |
32 |     if size.height == 0u32 {
33 |         return Result::Err(SizeError::ZeroHeight);
34 |     }
35 |
36 |     let pixels = size.area();
37 |
38 |     if pixels > max_pixels {
39 |         Result::Err(SizeError::TooManyPixels)
40 |     } else {
41 |         Result::Ok(pixels)
42 |     }
43 | }
44 |

```

TEXT

```

    | ... program continued ...
45 | fn main() {
46 |     let size = ImageSize::new(1920u32, 1080u32);
47 |     let max_pixels: UInt64 = 4_000_000u64;
48 |     let outcome = validate_size(&size, max_pixels);
49 |
50 |     match outcome {
51 |         Result::Ok(_) => println("image accepted"),
52 |         Result::Err(SizeError::ZeroWidth) =>
53 |             println("width must be positive"),
54 |         Result::Err(SizeError::ZeroHeight) =>
55 |             println("height must be positive"),
56 |         Result::Err(SizeError::TooManyPixels) =>
57 |             println("image is too large")
58 |     }
59 | }

```

Lines 24-27 are the contract for validation. A caller lends an `ImageSize`, copies a `UInt64` limit, and receives either a count or a typed failure. The body cannot quietly return a different error representation.

Lines 28-34 use early return to remove invalid zero dimensions before area calculation. Lines 36-42 calculate once and make the size-limit decision as a value-producing `if` expression.

Lines 45-48 assemble the call. Local inference derives the types of `size` and `outcome`; the limit remains explicitly annotated. `&size` is the visible ownership event.

Lines 50-58 consume the `Result` in an exhaustive `match`. Every arm calls `println`, which returns `Unit`, so the match also has type `Unit`. It is the final expression in `main`, consistent with `main`'s implicit `Unit` return.

The program is small, but its interfaces already carry several guarantees. A validation result cannot be mistaken for a bare count. The validator cannot mutate the size. The caller does not lose ownership. The match cannot silently forget a known error variant. Numeric representation changes are visible.

It also leaves important work to runtime and policy. Multiplication can overflow. The pixel limit comes from a constant literal rather than configuration. Width and height have structural but not semantic units. The program prints generic messages rather than returning an HTTP response. A production system would extend the program while preserving these basic contracts.

2.15 A disciplined way to read unfamiliar STARK

When you encounter a larger STARK file, read it in passes:

1. **Inventory the items.** Identify structs, enums, traits, implementations, functions, constants, imports, and modules.
2. **Read signatures before bodies.** Note which parameters are owned, borrowed, or mutably borrowed, and identify every return type.
3. **Locate boundary types.** Look for `Result`, `Option`, dynamic input types, and explicit conversions.
4. **Trace ownership events.** Mark moves, `&` borrows, and `&mut` borrows at call sites.
5. **Find block values.** Check the final expression and semicolon in each value-producing block.
6. **Expand paths mentally.** Distinguish associated items and variants using `::` from value methods and fields using `..`
7. **Check alternative handling.** Confirm that `match` arms cover the policy the enum expresses.
8. **Only then inspect implementation detail.** Arithmetic and control flow are easier to understand once contracts are known.

This method scales better than reading from the first token to the last and trying to remember every local fact. STARK is designed so that signatures and types reveal a large portion of a program's intended behavior.

Chapter summary

- A Core v1 source file is a sequence of top-level items; executable statements belong inside bodies.
- `main` uses ordinary function syntax and returns `Unit` when no return type is written.
- Local bindings are immutable by default; `mut` makes mutation explicit.
- Local types may be inferred, while function parameter and non-`Unit` return types are explicit.
- A block's unterminated final expression supplies its value. A semicolon discards an expression result.
- Structs define product types, enums define alternatives, and `impl` blocks attach associated functions and methods.
- `&T`, `&mut T`, and owned `T` parameters communicate different lifecycle contracts.
- `Result<T, E>` makes success and failure part of the type, and exhaustive `match` makes handling visible.
- `::` navigates paths and associated items; `.` starts from a value for fields and methods.
- Diagnostics should identify the violated rule, source provenance, actual and expected facts, and useful repairs.

Exercises

1. Change the successful `match` arm to bind the pixel count. What type does the new binding have?
2. Add a `TooNarrow` variant carrying the minimum accepted width. Which match sites must change?
3. Remove the `&` from `validate_size(&size, max_pixels)` and change the parameter accordingly. What happens to ownership of `size`?
4. Add a semicolon after `width * height` in `area`. Explain the resulting type mismatch without referring to the diagnostic text.
5. Rewrite `ImageSize::new` using struct-field shorthand.
6. Make `area` take `self` instead of `&self`. Which later uses of an `ImageSize` would become invalid after an `area` call?
7. Deliberately annotate `max_pixels` as `UInt32` while keeping the `u64` literal. Sketch the primary and related locations a useful diagnostic should show.

Source notes

1. Program structure, functions, blocks, items, expressions, patterns, paths, and semicolon behavior follow `STARKLANG/docs/spec/02-Syntax-Grammar.md`.
2. Keywords, identifiers, literals, comments, whitespace, and reserved tokens follow `STARKLANG/docs/spec/01-Lexical-Grammar.md`.
3. Primitive types, local inference, structs, enums, references, functions, and ownership typing follow `STARKLANG/docs/spec/03-Type-System.md`.
4. Scope resolution, type checking, borrow analysis, control flow, exhaustiveness, mutability, initialization, and diagnostic requirements follow `STARKLANG/docs/spec/04-Semantic-Analysis.md`.
5. `Result`, `println`, `Display`, and other prelude surfaces follow `STARKLANG/docs/spec/06-Standard-Library.md`.
6. Current implementation status and the Core-first roadmap follow `README.md`, `STARKLANG/docs/ROADMAP.md`, and `STARKLANG/docs/PLAN.md`.